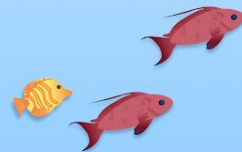


Wakacyjne Wyzwanie

Z  KN Solvro

Wykład 0

Wprowadzenie



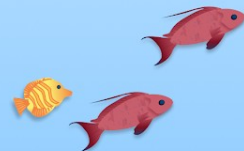
Witam na kursie DevOps Wakacyjnego Wyzwania!



- **Konrad Szczepaniak**
 - Przewodniczący DevOps w KN Solvro
 - OpenSourceowiec poza Solvro
 - SponsorBlock/DeArrow
 - Sable (klient [matrix]a)
 - aktualizacje pakietów Alpine Linux
 - Własna strona i serwisy:
<https://minibomba.pro>



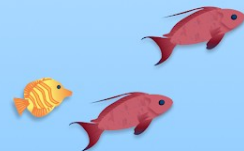
o prowadzeniu kursu



- Wszystkie wykłady będą prowadzone przeze mnie
- Prezentacje będą udostępniane w repozytorium
 - Publikowane na licencji CC BY-NC 4.0
- Wykłady będą nagrywane, nagrania publikowane
 - Publiczność będzie nagrywana na osobną ścieżkę
 - Zostanie usunięta przed opublikowaniem



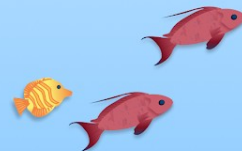
0 kursie



- 7 wykładów, wszystkie skupione na Linuxie
 1. Podstawy sieci (dzisiaj)
 2. Firewall i Policy-based routing
 3. Wirtualizacja i konfiguracja maszyny wirtualnej
 4. Wprowadzenie do konteneryzacji
 5. „Bardziej zaawansowana” konteneryzacja
 6. Monitoring: Grafana i Prometheus
 7. Konfiguracja VPN: Wireguard



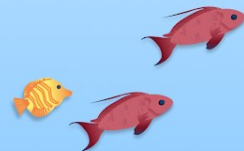
o kursie



- Do wykonywania zadań przyda się podstawowa znajomość Linuxa
 - Głównie korzystanie z terminala
 - Przechodzenie między katalogami, tworzenie/usuwanie plików i katalogów, edycja plików, podnoszenie sobie uprawnień
 - Znajomość założeń też nie zaszkodzi
 - Struktura systemu plików – czyli co gdzie jest
 - „wszystko jest plikiem”
 - UID 0 = root = może robić co chce



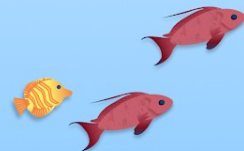
0 kursie



- Do wykonywania zadań będzie potrzebny system z zainstalowanym Linuxem
 - Dowolna dystrybucja, byle aktualna.
 - Dla początkujących: Debian 13
 - Dla ambitnych/zaawansowanych: Arch Linux :)
- **Co najmniej 2 listy będą wymagały Linuxa zainstalowanego bezpośrednio na dysku!**
 - Wykład 3 i 5 + hackaton?
 - Dla osób na Windowsie: polecam instalować na **osobnym dysku!**
- Pozostałe zadania da się wykonać w VMce (byle wydajnej)
 - WSLa nie testowałem i nie wspieram



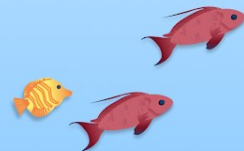
● uczestnictwie



- Z każdego wykładu będzie opublikowana lista zadań
 - Będą do zrobienia na ~2-4h
 - ▶ Jak ktoś kiedyś temat dany przerabiał, to pewnie da radę szybciej
- **Listy należy rozwiązać, sporządzić raport i przesłać go na maila**
 - W raporcie ważna treść i by był czytelny
 - ▶ Przebieg + screenshots + wolne wnioski
 - ▶ Nie trzeba specjalnie formatować
 - ▶ Przesłać jako PDF
 - Adres email do przesyłania raportów jest na #devops-info



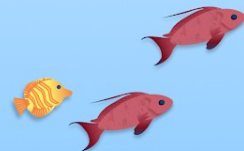
● uczestnictwie



- Zadania odsyłajcie do następnego wykładu
 - Ewentualnie piszcie, że się nie wyrobicie
- Odsyłając zadanie wrzućcie „[WW-x]” w tytuł
 - Podpisujcie się w mailach
- Nie używajcie LLMów – pytajcie na kanałach



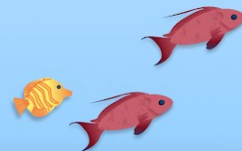
Zaliczenie



- By zaliczyć ścieżkę, trzeba odesłać wszystkie raporty **do 26 września**
- Udziałem w hackatonie można zaliczyć jeden raport
 - Hackaton będzie **12 września**



Dobre źródła



Dla Linuxowych
komend:
man <komenda>
(przykład: man podman)

```
~: man — Konsola
Plik  Edycja  Widok  Zakładki  Wtyczki  Ustawienia  Pomoc
Nowa karta  Podziel widok  Zezwól na śledzenie myszy
podman(1)  General Commands Manual  podman(1)

NAME
  podman - Simple management tool for pods, containers and images

SYNOPSIS
  podman [options] command

DESCRIPTION
  Podman (Pod Manager) is a fully featured container engine that is a simple daemonless tool. Podman provides a Docker-CLI comparable command line that eases the transition from other container engines and allows the management of pods, containers and images. Simply put: alias docker=podman. Most Podman commands can be run as a regular user, without requiring additional privileges.

  Podman uses Buildah(1) internally to create container images. Both tools share image (not container) storage, hence each can use or manipulate images (but not containers) created by the other.

  Default settings for flags are defined in containers.conf. Most settings for Remote connections use the server's containers.conf, except when documented in man pages.

  To manage containers, pods, volumes, networks, and images declaratively via systemd, use quadlet files. See podman-quadlet(1) and podman-systemd.unit(5).

  podman [GLOBAL OPTIONS]

GLOBAL OPTIONS
  --cdi-spec-dir=path
    The CDI spec directory path (may be set multiple times). Default path is /etc/cdi.

  --cgroup-manager=manager
    The CGroup manager to use for container cgroups. Supported values are cgroups or systemd. Default is systemd unless over-
  Manual page podman(1) line 1 (press h for help or q to quit)
```



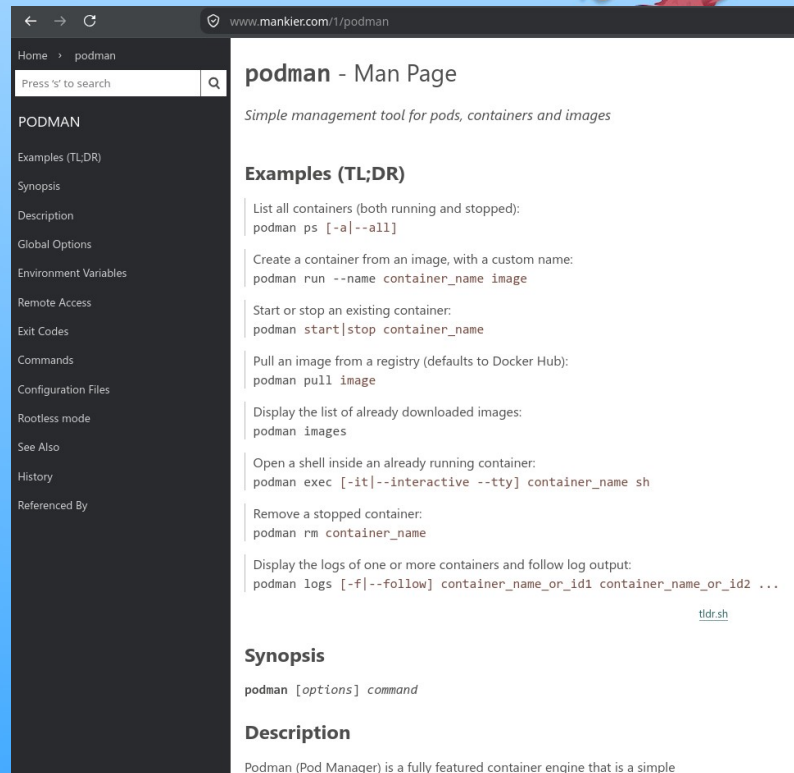
Dobre źródła

Jeżeli ktoś woli strony internetowe:

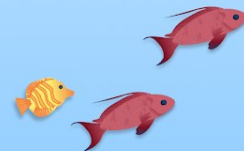
mankier.com

(znowu podman dla przykładu)

(jest wiele innych stron publikujących linuxowe manuale jako strony)



Dobre źródła



Oficjalna dokumentacja (szukajcie „<soft> docs”) (przykład: docker)

The screenshot shows the Docker documentation website. The left sidebar lists various Docker commands, with 'docker container run' highlighted. The main content area is titled 'docker container run' and includes a description, usage, aliases, and a table of options. The right sidebar contains a 'Table of contents' for the 'docker container run' page.

docker container run

Description
Create and run a new container from an image

Usage
`docker container run [OPTIONS] IMAGE [COMMAND] [ARG...]`

Aliases
`docker run`

Description
The `docker run` command runs a command in a new container, pulling the image if needed and starting the container.

You can restart a stopped container with all its previous changes intact using `docker start`. Use `docker ps -a` to view a list of all containers, including those that are stopped.

Options

Option	Default	Description
<code>--add-host</code>		Add a custom host-to-IP mapping (host:ip)

Table of contents

- Description
- Options
- Examples
 - Assign name (`--name`)
 - Capture container ID (`--cidfile`)
 - PID settings (`--pid`)
 - Example: run http inside a container
 - Example, join another container's PID namespace
 - Disable namespace remapping for a container (`--userns`)
 - UTS settings (`--uts`)
 - IPC settings (`--ipc`)
 - Escalate container privileges (`--privileged`)
 - Set working directory (`--workdir`)
 - Set storage driver options per container (`--storage-opt`)
 - Mount tmpfs (`--tmpfs`)
 - Mount volume (`-v`)
 - Mount volume read-only (`--read-only`)
 - Add bind mounts or volumes using the `--mount` flag
 - Publish or expose port (`-p`, `--expose`)
 - Publish all exposed ports (`-P`, `--publish-all`)
 - Set the pull policy (`--pull`)
 - Set environment variables (`-e`, `--env`, `--env-file`)
 - Set metadata on container (`-l`, `--label`, `--label-file`)



Dobre źródła

Wiki Archa

wiki.archlinux.org

(przykład: „Simple stateful firewall”)

wiki.archlinux.org/title/Simple_stateful_firewall

2.4 The INPUT chain

Similar to the previous chains, we set the default policy for the **INPUT** chain to **DROP** in case something somehow slips by our rules. Dropping all traffic and specifying what is allowed is the best way to make a secure firewall.

Warning

If you are logged in via SSH, the following will immediately disconnect the SSH session. To avoid it:

1. add the first INPUT chain rule below (it will keep the session open),
2. add a regular rule to allow inbound SSH (to be able to reconnect in case of a connection drop) and
3. set the policy.

```
# iptables -P INPUT DROP
```

Every packet that is received by any network interface will pass the **INPUT** chain first, if it is destined for this machine. In this chain, we make sure that only the packets that we want are accepted. For a simplified ASCII art showing how packets traverse those builtin chains, see [How Packets Traverse The Filters](#).

The first rule added to the INPUT chain will allow traffic that belongs to established connections, or new valid traffic that is related to these connections such as ICMP errors, or echo replies (the packets a host returns when pinged). ICMP stands for Internet Control Message Protocol. Some ICMP messages are very important and help to manage congestion and MTU, and are accepted by this rule:

```
# iptables -A INPUT -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
```

The connection state `ESTABLISHED` implies that either another rule previously allowed the initial (`--ctstate NEW`) connection attempt or the connection was already active (for example an active remote SSH connection).

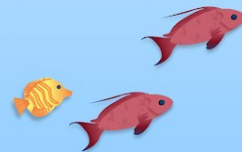
The second rule will accept all traffic from the “loopback” (lo) interface, which is necessary for many applications and services.

```
# iptables -A INPUT -i lo -j ACCEPT
```

Note

You can add more trusted interfaces here such as “eth1” if you do not want/need the traffic filtered by the firewall, but be warned that if you have a NAT setup that redirects any kind of traffic to this interface from anywhere else in the network (let us say a router), it will get through, regardless of any other settings you may have.





Pytania?

(zanim przejdziemy do wykładu 1)

